

Encuentro 4 — Primer programa y variables

1 — Fundamentos de Python y control del flujo

1. Metadatos de bloque

Campo	Detalle
Encuentro	4 de 36
Unidad	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Conceptual
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Primer programa en Python; variables, asignación y tipos básicos; operadores aritméticos; comentarios
Requisitos previos	Ninguno: es el primer encuentro con Python
Uso de celular	Permitido solo para consultar documentación o grabar fragmentos
Organización del trabajo	Individual con puesto fijo; se alterna entre escuchar, escribir código y resolver ejercicios

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Escribir y ejecutar un primer programa en Python desde VS Code y la terminal.
2. Declarar variables con nombres significativos en inglés y asignarles valores de tipo `int`, `float`, `str` y `bool`.
3. Usar los operadores aritméticos (`+`, `-`, `*`, `/`, `//`, `%`, `**`) en expresiones sencillas.
4. Agregar comentarios que documenten el propósito de cada bloque de código.

3. Teoría mínima (20 min)

Charla rápida: la caja de herramientas

Imaginate que abris una caja de herramientas. Cada herramienta tiene un nombre y un propósito: un martillo no es lo mismo que un destornillador, pero ambas son herramientas. En Python pasa algo parecido: cada valor que el programa necesita recordar se guarda en una **variable**, que tiene un nombre y un **tipo** que dice qué clase de dato contiene. Python te deja ver qué hay dentro con `print()` y hacer cuentas con operadores aritméticos, igual que una calculadora.

Lo mínimo indispensable

Un primer programa. La forma más simple de que Python haga algo es con `print()`:

```
# Muestra un mensaje en la consola
print("Hola, mundo!")
```

Variables. Una variable es un nombre que le ponés a un valor. El operador `=` asigna (guarda) el valor de la derecha en la variable de la izquierda:

```
# Tipos basicos
age = 17          # int (entero)
price = 12.5      # float (real)
name = "Ana"      # str (texto, siempre con comillas dobles)
active = True     # bool (logico: True o False)

print(age)
print(price)
print(name)
print(active)
```

Los nombres de variable se escriben en **inglés**, en **snake_case** (todo minúscula, separado por guión bajo), sin tildes ni eñes.

Operadores aritméticos. Python entiende las cuentas como una calculadora:

```
a = 10
b = 3

suma = a + b      # 13
resta = a - b     # 7
multi = a * b     # 30
div_real = a / b  # 3.333... (siempre float)
div_entera = a // b # 3      (trunca decimales)
resto = a % b     # 1       (modulo)
potencia = a ** b # 1000    (10 elevado a 3)

print(suma)
print(resto)
print(potencia)
```

Comentarios. Todo lo que está después de `#` hasta el final de la línea es un comentario: Python lo ignora. Sirve para explicar qué hace cada parte del código.

4. Práctica guiada (35 min)

Vamos a escribir un programa que calcule cuántos días, horas y minutos vivió una persona hasta cierto cumpleaños, usando solo los datos del año actual y su edad. Abrí VS Code, creé un archivo `edad.py` y escribí lo siguiente:

```

# Calcula tiempo vivido aproximado en días, horas y minutos
# Usa solo valores fijos (sin entrada de datos todavía)

name = "Luis"
age = 17          # edad actual en años
days_per_year = 365
hours_per_day = 24
minutes_per_hour = 60

# Cálculos aproximados (sin contar bisieños)
total_days = age * days_per_year
total_hours = total_days * hours_per_day
total_minutes = total_hours * minutes_per_hour

print("Nombre:")
print(name)
print("Edad en años:")
print(age)
print("Días vividos (aprox):")
print(total_days)
print("Horas vividas (aprox):")
print(total_hours)
print("Minutos vividos (aprox):")
print(total_minutes)

```

Salida esperada (los valores numéricos dependen de los cálculos):

```

Nombre:
Luis
Edad en años:
17
Días vividos (aprox):
6205
Horas vividas (aprox):
148920
Minutos vividos (aprox):
8935200

```

Ahora modifiquen el programa para que: 1. Cambien el valor de `name` por su propio nombre. 2. Cambien `age` por su propia edad. 3. Agreguen una variable `weeks_per_year = 52` y calculen las semanas vividas (`total_weeks`). 4. Muestren también las semanas vividas con `print()`.

Pista: la multiplicación se escribe con `*`; el orden de las operaciones es el mismo que en matemática (primero multiplicación y división, después suma y resta). Usá paréntesis si querés asegurar el orden.

Ejecutá el programa con `python edad.py` desde la terminal integrada de VS Code.

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `rectangulo.py` que calcule el área y el perímetro de un rectángulo. Los lados del rectángulo se guardan como variables al principio del código. El programa debe mostrar en la consola el valor de cada lado, el área calculada y el perímetro calculado.

Usá: - Largo = 12.5 (tipo `float`) - Ancho = 8.3 (tipo `float`)

Pista: el área es largo × ancho. El perímetro es 2 × largo + 2 × ancho. Recordá que los identificadores van en inglés.

6. Extensión y consolidación (20 min)

Si terminaste el ejercicio anterior, probá estas variaciones:

1. **Redondear con operadores.** Usá `//` para calcular cuántas veces enteras entra el ancho en el largo.
2. **Potencia.** Calculá el área de un cuadrado cuyo lado es el mismo valor del largo (12.5) usando el operador `**`.
3. **Mezcla de tipos.** Asigná `count = 10` (int) y `unit_price = 4.99` (float), calculá `total = count * unit_price` y mostrá el resultado. ¿De qué tipo es `total`?

7. Cierre (10 min)

Qué te llevás

- Python ejecuta instrucciones de arriba a abajo.
- Las variables guardan valores en memoria con un nombre y un tipo.
- Cada tipo (`int`, `float`, `str`, `bool`) sirve para una clase de dato distinta.
- Los operadores aritméticos funcionan como en una calculadora, pero `/` siempre devuelve `float` y `//` trunca.
- Los comentarios con `#` explican el código sin afectar la ejecución.

Lo que viene

En el **Encuentro 5 — Entrada y salida por consola** vamos a hacer que el programa hable con el usuario: leer datos con `input()`, convertirlos al tipo correcto y armar mensajes con f-strings. Ahí tus programas van a dejar de ser fijos y van a reaccionar a lo que el usuario escribe.

8. Errores comunes y trampas

Error	Causa	Fix
<code>SyntaxError: invalid syntax</code> al escribir <code>print "hola"</code>	En Python 3 los paréntesis en <code>print()</code> son obligatorios	Escribir <code>print("hola")</code>
<code>NameError: name 'edad' is not defined</code>	La variable se llama <code>age</code> (inglés) pero se usa <code>edad</code> (español)	Usar siempre el nombre exacto que se asignó; mantener identificadores en inglés
<code>7 / 2</code> da <code>3.5</code> en vez de <code>3</code>	El operador <code>/</code> es división real, no entera	Usar <code>//</code> si se quiere división entera (<code>7 // 2</code> → <code>3</code>)
Asignar <code>miVariable = 5</code> sin declaración previa es válido pero confunde	No hay error, pero mezclar estilos (camelCase con snake_case) dificulta la lectura	Usar siempre <code>snake_case</code> (<code>my_variable</code>)
Olvidar las comillas en un texto: <code>name = Ana</code>	Sin comillas, Python busca una variable llamada <code>Ana</code> que no existe	Escribir <code>name = "Ana"</code> con comillas dobles
<code>a = a + 1</code> antes de haber definido <code>a</code>	No se puede usar una variable antes de asignarle un valor	Primero asignar el valor inicial (<code>a = 0</code>)

Encuentro 5 — Entrada y salida por consola

1 — Fundamentos de Python y control del flujo

1. Metadatos de bloque

Campo	Detalle
Encuentro	5 de 36
Unidad	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Entrada con <code>input()</code> , salida con <code>print()</code> , conversión de tipos (<code>int()</code> , <code>float()</code> , <code>str()</code>) , f-strings
Requisitos previos	Encuentro 4: tipos básicos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>) , operadores aritméticos, comentarios
Uso de celular	Permitido solo para consultar documentación
Organización del trabajo	Individual con monitores rotativos: un compañero observa mientras el otro escribe

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

- Capturar datos del usuario con `input()` y entender que siempre devuelve texto (`str`).
- Convertir texto a número con `int()` y `float()` dentro de un `try/except ValueError` .
- Armaz mensajes de salida con f-strings en vez de concatenar con `+` .
- Mostrar valores numéricos con formato en pantalla usando f-strings.

3. Teoría mínima (20 min)

Charla rápida: el programa que escucha

Hasta ahora el programa hablaba pero no escuchaba. Es como un recepcionista que dice siempre la misma frase sin preguntar tu nombre. Con `input()` el programa hace una pausa, lee lo que el usuario tipea y lo guarda en una variable. Pero hay una trampa: `input()` siempre devuelve texto, aunque el usuario haya tipeado un número. Si querés sumar, primero tenés que convertir ese texto a número con `int()` o `float()`.

Lo mínimo indispensable

Leer datos con `input()`. La función `input()` muestra un mensaje (opcional) y espera que el usuario escriba algo y presione Enter. Lo que sea que escriba, lo devuelve como texto (`str`).

```
# input() siempre devuelve str
name = input("Como te llamas? ")
print("Hola,")
print(name)
```

Convertir tipos. Si el usuario tipea `"17"`, eso es texto, no número. Para poder hacer cuentas hay que convertirlo:

```
age_text = input("Cuantos años tenes? ")
age = int(age_text)      # convierte "17" -> 17
next_year = age + 1
print("El año que viene vas a tener:")
print(next_year)
```

Pero si el usuario tipea algo que no se puede convertir (por ejemplo `"diecisiete"`), `int()` lanza un error `ValueError` y el programa se rompe. Vamos a ver cómo manejarlo desde el Encuentro 6; por ahora conviene tipear bien los datos.

f-strings. Armar mensajes con `print("Edad:", age)` funciona, pero es más cómodo usar f-strings: ponés una `f` antes de las comillas y escribís las variables entre llaves `{}`:

```
name = input("Como te llamas? ")
age = int(input("Cuantos años tenes? "))

# f-string: las variables van entre {}
print(f"Hola {name}, el año que viene vas a tener {age + 1} años.")
```

Las f-strings también permiten dar formato a números:

```
price = 1250.5
# :.2f muestra 2 decimales
print(f"El precio es ${price:.2f}")
```

4. Práctica guiada (35 min)

Vamos a escribir un programa `calculadora_edad.py` que lea el nombre y el año de nacimiento del usuario y calcule su edad aproximada.

```
# Calcula la edad aproximada a partir del año de nacimiento
# Usa el año actual como valor fijo (después vamos a mejorarlo)

# Entrada de datos
name = input("Como te llamas? ")
birth_year_text = input("En que año naciste? ")

# Conversion de str a int
birth_year = int(birth_year_text)

# Año actual (fijo para el ejemplo)
current_year = 2024

# Calculo
age = current_year - birth_year

# Salida con f-string
print(f"Hola {name}, este año cumplis (o ya cumpliste) {age} años.")
```

Salida esperada (con datos de ejemplo):

```
Como te llamas? Martina
En que año naciste? 2007
Hola Martina, este año cumplis (o ya cumpliste) 17 años.
```

Ahora modifiquen el programa para: 1. Pedir también el mes de nacimiento (como número) y el mes actual. 2. Mostrar un mensaje distinto si la persona ya cumplió años este año o todavía no.

Pista: usá `int(input(...))` en una sola línea para ahorrar variables intermedias.

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `promedio.py` que le pida al usuario tres notas (números con decimales) de una materia y muestre el promedio con **exactamente 2 decimales**.

El programa debe: 1. Pedir las tres notas una por una. 2. Convertirlas a `float`. 3. Calcular el promedio. 4. Mostrar el resultado con un mensaje como `"Tu promedio es 7.83"`.

Pista: usá `float(input(...))` para leer cada nota. Para mostrar 2 decimales usá `:.2f` adentro de la f-string.

6. Extensión y consolidación (20 min)

Si terminaste el promedio, probá estas variaciones:

- Promedio ponderado.** Pedí también el peso (porcentaje) de cada nota. El promedio ponderado es `(n1*p1 + n2*p2 + n3*p3) / (p1+p2+p3)`.
- Redondeo sin formato.** Calculá el promedio y mostralo con `print(round(promedio, 2))`. ¿Se ve igual que con f-string?
- Nombre completo.** Pedí nombre y apellido por separado y armá un mensaje que diga `"Alumno: Garcia, Martina – Promedio: 7.83"`.

7. Cierre (10 min)

Qué te llevás

- `input()` lee todo como texto (`str`), aunque el usuario tipee números.
- Para hacer cuentas hay que convertir con `int()` (entero) o `float()` (decimal).

- Si la conversión falla, `ValueError` rompe el programa.
- Las f-strings (`f"..."`) arman mensajes legibles sin concatenar con `+`.
- `:.2f` adentro de una f-string controla cuántos decimales se muestran.

Lo que viene

En el **Encuentro 6 — Condicionales y operadores** vamos a aprender a tomar decisiones: si el usuario es mayor de edad, si la nota alcanza para aprobar, si un número es par o impar. Todo con `if`, `elif` y `else`.

8. Errores comunes y trampas

Error	Causa	Fix
<code>ValueError: invalid literal for int()</code> al ejecutar <code>int("diez")</code>	El usuario tipeó texto no numérico y <code>int()</code> no sabe convertirlo	Validar la entrada con <code>try/except ValueError</code> (lo vemos en el Encuentro 6)
<code>TypeError: can only concatenate str</code> al hacer <code>"Edad: " + age</code>	<code>age</code> es <code>int</code> , no se puede concatenar con <code>+</code> a un <code>str</code>	Usar f-string: <code>f"Edad: {age}"</code>
El programa termina sin esperar entrada	No se llamó a <code>input()</code> con un mensaje visible	Agregar <code>input("Mensaje: ")</code>
<code>input()</code> "devuelve" el número pero sigue siendo texto	No se convirtió con <code>int()</code> o <code>float()</code>	<code>age = int(input("Edad: "))</code>
La f-string muestra <code>{age}</code> literal en vez del valor	Falta la <code>f</code> antes de las comillas: se usó <code>"..."</code> en vez de <code>f"..."</code>	Escribir <code>f"Edad: {age}"</code>
<code>7 + 3</code> concatenado como <code>"73"</code> en vez de sumar <code>10</code>	Las variables son <code>str</code> (no se convirtieron al leer)	Convertir en el momento de la lectura: <code>int(input(...))</code>

Encuentro 6 — Condicionales y operadores

1 — Fundamentos de Python y control del flujo

1. Metadatos de bloque

Campo	Detalle
Encuentro	6 de 36
Unidad	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Operadores de comparación y lógicos (<code>and</code> , <code>or</code> , <code>not</code>); estructura <code>if</code> / <code>elif</code> / <code>else</code>
Requisitos previos	Encuentro 5: <code>input()</code> , conversión de tipos (<code>int()</code> , <code>float()</code>), f-strings
Uso de celular	Solo para consultar documentación o probar condiciones en repl.it
Organización del trabajo	Pares rotativos (programación en dúos, un teclado cada 15 min)

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Comparar valores usando `==` , `!=` , `<` , `>` , `<=` , `>=` y comprender que cada comparación devuelve `True` o `False` .
2. Combinar condiciones con `and` , `or` , `not` para expresar decisiones compuestas.
3. Escribir estructuras `if` / `elif` / `else` que ejecuten distintos bloques según una condición.
4. Validar entrada numérica con `try/except ValueError` para evitar que el programa se rompa con datos inválidos.

3. Teoría mínima (20 min)

Charla rápida: el semáforo

Un semáforo decide: si está verde, pasás; si está rojo, parás; si está amarillo, preparate. Tu programa necesita hacer lo mismo: **si** el usuario es mayor de edad, mostrará un mensaje; **si no**, mostrará otro. Las condiciones son preguntas de sí o no que el programa evalúa como `True` (verdadero) o `False` (falso). Con `and` y `or` podés hacer preguntas compuestas: “si tiene más de 15 **y** trajo permiso”.

Lo mínimo indispensable

Operadores de comparación. Cada comparación devuelve `True` o `False`:

```
age = 17
print(age == 18)    # False (es igual?)
print(age != 18)    # True  (es distinto?)
print(age < 18)     # True  (es menor que?)
print(age >= 18)    # False (es mayor o igual?)
```

Operadores lógicos. Combinan condiciones:

```
has_license = True
age = 20

# and: ambas deben ser True
print(age >= 18 and has_license) # True

# or: al menos una debe ser True
has_permission = False
print(age >= 18 or has_permission) # True

# not: invierte el valor
print(not has_license) # False
```

if / elif / else. La estructura que decide:

```
age = int(input("Que edad tenes? "))

if age >= 18:
    print("Sos mayor de edad.")
else:
    print("Sos menor de edad.")
```

Con más de dos caminos se usa `elif` (abreviatura de “else if”):

```
grade = float(input("Nota del examen: "))

if grade >= 8:
    print("Muy bien, aprobaste con excelente.")
elif grade >= 6:
    print("Aprobaste.")
elif grade >= 4:
    print("Aprobaste con lo justo.")
else:
    print("Desaprobaste.")
```

Validación con try/except. Si el usuario tipea letras donde esperamos números, `int()` o `float()` lanzan `ValueError`. Para que el programa no se rompa, se atrapa el error y se vuelve a pedir:

```

while True:
    try:
        age = int(input("Edad: "))
        break # sale del while si la conversion fue exitosa
    except ValueError:
        print("Eso no es un numero entero; intenta de nuevo.")

# Ahora age es un int seguro
if age >= 18:
    print("Mayor de edad.")
else:
    print("Menor de edad.")

```

4. Práctica guiada (35 min)

Escribamos un programa `clasificador_notas.py` que pida una nota, la valide y clasifique al alumno.

```

# Clasifica una nota numerica en categorias

while True:
    try:
        grade = float(input("Ingresa la nota (0 a 10): "))
        if grade >= 0 and grade <= 10:
            break
        else:
            print("La nota debe estar entre 0 y 10.")
    except ValueError:
        print("Eso no es un numero; intenta de nuevo.")

if grade >= 9:
    category = "Excelente"
elif grade >= 7:
    category = "Bueno"
elif grade >= 5:
    category = "Regular"
elif grade >= 3:
    category = "Insuficiente"
else:
    category = "Malo"

print(f"Nota: {grade:.1f} - Categoria: {category}")

```

Salida esperada (varios casos):

```

Ingresa la nota (0 a 10): 8.5
Nota: 8.5 - Categoria: Bueno

Ingresa la nota (0 a 10): once
Eso no es un numero; intenta de nuevo.
Ingresa la nota (0 a 10): 11
La nota debe estar entre 0 y 10.
Ingresa la nota (0 a 10): 4
Nota: 4.0 - Categoria: Insuficiente

```

Ahora modifiquen el programa: 1. Agreguen una condición con `or`: si la nota es menor que 2 o mayor que 9, mostrar un mensaje extra `"Caso extremo"`. 2. Agreguen la categoría "Sobresaliente" para nota 10 exacta (antes que "Excelente").

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `bisiesto.py` que pida un año (entero positivo) y determine si es bisiesto o no.

Un año es bisiesto si: - Es divisible por 4 **y** no es divisible por 100, **o** - Es divisible por 400.

Usá validación de entrada con `try/except ValueError`. Mostrá el resultado con un f-string.

Pista: para saber si un número es divisible por otro, usá el operador `%`. Si el resto es 0, es divisible. Por ejemplo: `anio % 4 == 0`.

6. Extensión y consolidación (20 min)

Si terminaste el bisiesto, probá estas variaciones:

1. **Tres números.** Pedí tres números y mostrá cuál es el mayor (sin usar `max()`). Usá `if` anidados o condiciones compuestas.
2. **Día de la semana.** Pedí un número del 1 al 7 y mostrá el nombre del día (lunes=1, domingo=7). Usá `if` / `elif` / `else` (no adelantes `match` ni diccionarios).
3. **Calculadora simple.** Pedí dos números y una operación (`+`, `-`, `*`, `/`). Según la operación, mostrá el resultado. Validá que no se divida por cero.

7. Cierre (10 min)

Qué te llevás

- Las comparaciones (`==`, `!=`, `<`, `>`, `<=`, `>=`) devuelven `True` o `False`.
- `and` exige que todas las condiciones sean `True`; `or` alcanza con una.
- `if` / `elif` / `else` elige qué bloque ejecutar según las condiciones.
- El orden de los `elif` importa: Python ejecuta el primero que da `True`.
- `try/except ValueError` evita que el programa se rompa con entrada inválida.

Lo que viene

En el **Encuentro 7 — Bucles for y while** vamos a hacer que el programa repita acciones automáticamente con bucles `for` y `while`. Vas a poder recorrer rangos de números, procesar texto carácter por carácter y repetir hasta que se cumpla una condición.

8. Errores comunes y trampas

Error	Causa	Fix
Poner <code>=</code> en vez de <code>==</code> en una condición: <code>if age = 18</code>	<code>=</code> es asignación, no comparación; Python lanza <code>SyntaxError</code>	Usar <code>==</code> en condiciones: <code>if age == 18</code>
Condición que nunca se cumple por orden incorrecto	<code>if grade >= 5</code> antes de <code>if grade >= 8</code> : el primer bloque atrapa notas <code>>= 5</code> antes de que llegue al de <code>>= 8</code>	Ordenar de más restrictiva a menos restrictiva
Olvidar los dos puntos <code>:</code> al final de <code>if</code> , <code>elif</code> o <code>else</code>	Python requiere <code>:</code> para abrir el bloque	Escribir <code>if condicion:</code>
Confundir <code>and</code> con <code>&</code> o <code>or</code> con <code> </code>	<code>&</code> y <code> </code> son operadores de bits, no lógicos booleanos	Usar <code>and</code> y <code>or</code> (palabras) para condiciones
Validar con <code>if edad > 17 and < 65</code> (falta la variable en la segunda parte)	Cada condición debe ser completa	Escribir <code>if edad > 17 and edad < 65</code>
<code>try</code> sin <code>except</code> o <code>except:</code> desnudo	El bloque <code>try</code> necesita un manejador; <code>except:</code> sin tipo atrapa todo, incluso errores que no debería	Usar <code>except ValueError:</code> o el tipo específico

Encuentro 7 — Bucles for y while

1 — Fundamentos de Python y control del flujo

1. Metadatos de bloque

Campo	Detalle
Encuentro	7 de 36
Unidad	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Bucles <code>for</code> sobre <code>range()</code> y strings; bucle <code>while</code> con condición de corte; contadores y acumuladores
Requisitos previos	Encuentro 6: <code>if / elif / else</code> , operadores de comparación y lógicos, <code>try/except ValueError</code>
Uso de celular	Solo para documentación
Organización del trabajo	Individual, con variaciones en dúo para el ejercicio de extensión

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Repetir acciones una cantidad conocida de veces con `for` y `range()` .
2. Recorrer un string carácter por carácter con `for` .
3. Repetir hasta que se cumpla una condición con `while` .
4. Usar contadores (contar repeticiones) y acumuladores (sumar o promediar valores).

3. Teoría mínima (20 min)

Charla rápida: la vuelta al circuito

Imaginá que tenés que correr 5 vueltas a una pista. No escribís “doy una vuelta” cinco veces: contás las vueltas con un número y cuando llegás a 5 terminás. El bucle `for` es como ese contador automático: sabés de antemano cuántas vueltas dar. El bucle `while` es distinto: es como esperar el colectivo — no sabés cuánto va a tardar, solo repetís “¿llegó?” hasta que efectivamente llegue.

Lo mínimo indispensable

for sobre range(). `range(n)` genera números de 0 a n-1. Con `for` iteramos sobre cada uno:

```
# Muestra los numeros del 0 al 4
for i in range(5):
    print(i)

# range(inicio, fin, paso)
print("Del 2 al 8, de a 2:")
for i in range(2, 9, 2):
    print(i)
```

for sobre strings. Un string es una secuencia de caracteres. Podés recorrerlo con `for` :

```
name = "Ana"
for letter in name:
    print(letter)

# A
# n
# a
```

Contadores y acumuladores. Patrones fundamentales:

```
# Contador: cuenta cuantas veces pasa algo
count = 0
for i in range(10):
    if i % 2 == 0:
        count = count + 1
print(f"Pares del 0 al 9: {count}")

# Acumulador: suma valores
total = 0
for i in range(1, 6):
    total = total + i
print(f"Suma del 1 al 5: {total}")
```

while. Se ejecuta mientras la condición sea `True`. Hay que asegurarse de que la condición cambie dentro del bloque, o el bucle nunca termina:

```
# Cuenta regresiva con while
countdown = 5
while countdown > 0:
    print(countdown)
    countdown = countdown - 1 # sin esta linea, el bucle es infinito
print("Despegue!")
```

while True con break. Es común usarlo para validar entrada:

```

while True:
    try:
        age = int(input("Edad: "))
        if age > 0:
            break
        print("La edad debe ser positiva.")
    except ValueError:
        print("Eso no es un numero.")

```

4. Práctica guiada (35 min)

Escribamos un programa `suma_rango.py` que pida dos números (inicio y fin) y calcule la suma de todos los enteros entre ellos (inclusive).

```

# Suma todos Los enteros entre inicio y fin (inclusive)

# Validacion del inicio
while True:
    try:
        start = int(input("Ingresa el numero de inicio: "))
        break
    except ValueError:
        print("Eso no es un numero entero.")

# Validacion del fin
while True:
    try:
        end = int(input("Ingresa el numero de fin: "))
        break
    except ValueError:
        print("Eso no es un numero entero.")

# Acumulador: suma todos los numeros del rango
total = 0
for num in range(start, end + 1):
    total = total + num

print(f"La suma de {start} a {end} es {total}.")

```

Salida esperada:

```

Ingresa el numero de inicio: 1
Ingresa el numero de fin: 5
La suma de 1 a 5 es 15.

Ingresa el numero de inicio: 10
Ingresa el numero de fin: 10
La suma de 10 a 10 es 10.

```

Ahora modifiquen el programa: 1. Cuenten cuántos números hay entre inicio y fin (pista: ya lo saben con `end - start + 1`). 2. Sumen solo los pares (usen `% 2 == 0` adentro del `for`). 3. Muestren la cantidad de pares y la cantidad de impares encontrados.

5. Ejercicio independiente (25 min)

Consigna: Escribí un programa `adivina.py` que elija un número secreto fijo (por ejemplo 7) y le pida al usuario que adivine. El programa da pistas ("más alto", "más bajo") y cuenta cuántos intentos usó. Cuando acierta, muestra un mensaje de felicitaciones con la cantidad de intentos.

Requisitos: - El número secreto se asigna como variable al inicio (`secret = 7`). - El programa usa un `while` que sigue hasta que el usuario acierta. - Validación con `try/except ValueError`. - Contador de intentos.

Pista: el `while` se corta cuando la entrada del usuario es igual al secreto. Usá un contador `attempts = 0` y sumale 1 en cada vuelta.

6. Extensión y consolidación (20 min)

Si terminaste el juego de adivinar, probá estas variaciones:

1. **Rango variable.** Al iniciar el programa, pedí un límite inferior y superior. El número secreto se calcula con `(limite_inferior + limite_superior) // 2` (la mitad del rango, siempre entero).
2. **Contar vocales.** Pedí una palabra y contá cuántas vocales tiene (a, e, i, o, u). Recorrela con `for letter in word:`.
3. **Tabla de multiplicar.** Pedí un número y mostrá su tabla del 1 al 10 con un `for`. Cada línea: `"5 x 3 = 15"`.

7. Cierre (10 min)

Qué te llevás

- `for` itera sobre una secuencia: `range(n)` da 0 a n-1, `range(inicio, fin, paso)` da control preciso.
- `while` repite mientras la condición sea `True`; si la condición nunca cambia, es un bucle infinito.
- Los contadores aumentan de 1 en 1 (`count = count + 1`); los acumuladores suman valores (`total = total + valor`).
- `while True` con `break` adentro es el patrón estándar para validar entrada hasta que sea correcta.

Lo que viene

En el **Encuentro 8 — Cierre U1: repaso y TP** vamos a cerrar la Unidad 1 con un repaso general y el trabajo sobre el TP-U1. También vamos a aprender el ciclo completo de entrega con Git y GitHub, que vas a usar para el resto del curso.

8. Errores comunes y trampas

Error	Causa	Fix
Bucle infinito: <code>while intentos < 5:</code> sin modificar <code>intentos</code> adentro	La condición nunca cambia; Python no corta solo	Agregar <code>intentos = intentos + 1</code> dentro del bloque
<code>for i in range(1, 5)</code> espera que incluya el 5	<code>range(1, 5)</code> genera 1, 2, 3, 4; el fin es exclusivo	Usar <code>range(1, 6)</code> para incluir el 5
Usar <code>for i in range(5):</code> y después tratar a <code>i</code> como el valor acumulado	<code>i</code> toma cada valor de la secuencia, no es un acumulador	Usar una variable separada como <code>total = total + i</code>
Olvidar la conversión en <code>int(input(...))</code> dentro de un <code>while True</code>	<code>input()</code> siempre devuelve <code>str</code> ; la comparación numérica falla	Convertir apenas se lee: <code>int(input(...))</code>
<code>while True:</code> sin <code>break</code> en ninguna rama	El bucle no tiene salida	Agregar <code>break</code> cuando la condición de corte se cumple
<code>total = total + 1</code> cuando querés contar pasos	<code>total = total + 1</code> cuenta (contador), <code>total = total + valor</code> acumula (acumulador)	Usar <code>count = count + 1</code> para contar, y otra variable para sumar valores

Encuentro 8 — Cierre U1: repaso y TP

1 — Fundamentos de Python y control del flujo · Encuentro de cierre de unidad

1. Metadatos de bloque

Campo	Detalle
Encuentro	8 de 36
Unidad	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Actitudinal
Estructura	cierre
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Sistematización de U1; TP-U1 (kiosco escolar); ciclo de entrega con Git y GitHub
Requisitos previos	Encuentro 7: bucles for y while , contadores, acumuladores
TP obligatorio	TP-U1: programa de consola con entrada y control de flujo
Uso de celular	Solo para acceder a GitHub
Organización del trabajo	Grupos de 2 (quedan fijos para todo el curso); cada grupo usa una computadora

Reparto de tiempos

Momento	Tiempo
Apertura	10 min
Consolidación	40 min
Trabajo del TP	45 min
Ciclo de entrega	15 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Sistematizar los conceptos de la Unidad 1: variables, tipos, entrada/salida, condicionales y bucles.
2. Escribir un programa integrador que combine entrada, validación, condicionales y bucles en un único archivo.
3. Inicializar un repositorio Git local con .gitignore , hacer commits y conectar con un repositorio remoto en GitHub.
4. Subir el TP-U1 terminado a GitHub siguiendo el ciclo de entrega.

3. Apertura y puente (10 min)

Recorrida rápida de lo que vimos en los encuentros anteriores:

- **E4:** print(), variables, operadores aritméticos.
- **E5:** input(), conversión de tipos, f-strings.
- **E6:** if/elif/else, operadores de comparación y lógicos, try/except.
- **E7:** for con range(), while, contadores y acumuladores.

Hoy juntamos todo en un solo programa: el TP-U1. Además aprendemos a entregarlo con Git y GitHub.

4. Consolidación (40 min)

Sistematización: cómo se arma un programa completo

Entre los 4 encuentros anteriores vimos estos bloques. Hoy los vamos a usar todos juntos:

1. **Entrada con validación** (`while True` + `try/except`)
2. **Menú con condicionales** (`if` / `elif` / `else`)
3. **Bucles que repiten hasta que el usuario quiere salir** (`while` con bandera)
4. **Acumuladores y contadores** para llevar cuentas

Vamos a ver el esqueleto de un programa con menú (sin el tema del TP todavía):

```
# Esqueleto de programa con menu
# El usuario elige opciones hasta que selecciona "Salir"

print("=== SISTEMA DE GESTION ===")
print("1. Ver datos")
print("2. Agregar item")
print("3. Salir")

option = input("Elegi una opcion: ")

if option == "1":
    print("Mostrando datos...")
elif option == "2":
    print("Agregando item...")
elif option == "3":
    print("Saludos!")
else:
    print("Opcion invalida.")
```

El problema de ese esqueleto es que se ejecuta una sola vez. ¿Cómo hacemos para que el menú se muestre hasta que el usuario elija “Salir”? Con un bucle:

```
# Programa que repite el menu hasta que el usuario elige Salir

running = True

while running:
    print()
    print("=== SISTEMA DE GESTION ===")
    print("1. Ver datos")
    print("2. Agregar item")
    print("3. Salir")

    option = input("Elegi una opcion: ")

    if option == "1":
        print("Mostrando datos...")
    elif option == "2":
        print("Agregando item...")
    elif option == "3":
        running = False
        print("Saludos!")
    else:
        print("Opcion invalida.")
```

Esa estructura de `while` con bandera (`running = True` / `running = False`) es la que vas a usar en el TP.

5. Trabajo del TP (45 min)

Consigna del TP-U1

TP-U1: programa de consola con entrada y control de flujo

Grupos de 2. Escribir un programa `kiosco.py` que simule la atención en un kiosco escolar.

El programa debe: 1. Mostrar un menú principal con estas opciones: - **1 — Ver lista de productos** (muestra productos, precios y stock fijos desde variables). - **2 — Registrar una venta** (pide un código de producto y una cantidad, valida stock, descuento y muestra el total con descuento eventual). - **3 — Mostrar resumen de ventas** (total de unidades vendidas en la sesión y dinero acumulado). - **4 — Salir** (cierra el programa).

2. Definir al menos 4 productos al inicio del programa, cada uno con: código (int), nombre (str), precio (float) y stock (int). Usar listas paralelas o un diccionario (lo que quede más claro).

3. Para la opción **Registrar una venta**:

- Pedir código de producto.
- Validar que exista (si no, mostrar error y volver al menú).
- Pedir cantidad.
- Validar stock suficiente (si no, mostrar error).
- Si la cantidad supera las 5 unidades, aplicar un descuento del 10%.
- Descontar del stock.
- Acumular en el total de ventas del día.

4. El programa corre hasta que el usuario elige “Salir”.

5. Toda entrada numérica debe validarse con `try/except ValueError`.

Pista: usá un diccionario para los productos, donde la clave es el código y el valor es una tupla con (nombre, precio, stock). Por ejemplo:

```
products = {  
    101: ("Caramelos", 50.0, 30),  
    102: ("Chicles", 25.0, 50),  
    103: ("Galletitas", 100.0, 20),  
    104: ("Jugos", 150.0, 15)  
}
```

Después, para el resumen, llevá dos acumuladores: `total_units` y `total_money`.

6. Ciclo de entrega (15 min)

Hoy aprendemos el ciclo completo de entrega con Git y GitHub. Este ciclo lo vas a repetir en cada TP del curso.

Paso 1: Crear el repositorio en GitHub.

1. Entrá a github.com e iniciá sesión con la cuenta del grupo.
2. Creá un repositorio nuevo con nombre `tp-lap`.
3. **No** marques "Initialize with README". Dejalo vacío.

Paso 2: Inicializar Git local.

```
# En la terminal, dentro de la carpeta del TP  
cd ruta/al/tp-u1  
  
# Inicializar repositorio  
git init  
  
# Crear .gitignore para evitar basura  
echo "__pycache__/" > .gitignore  
echo ".vscode/" >> .gitignore
```

Paso 3: Primer commit.

```
# Agregar todo al area de preparacion  
git add .  
  
# Hacer el primer commit  
git commit -m "tp-u1: version inicial del programa"
```

Paso 4: Conectar con GitHub y subir.

```
# Agregar el repositorio remoto (reemplazar URL con la de tu repo)  
git remote add origin https://github.com/tu-usuario/tp-lap.git  
  
# Subir la rama main  
git push -u origin main
```

A partir de ahora, cada vez que terminen un avance importante:

```
git add .  
git commit -m "tp-u1: <descripcion del avance>"  
git push
```

7. Cierre (10 min)

Qué te llevás

- Un programa completo combina: entrada, validación, condicionales, bucles y acumuladores.
- El menú con `while` + bandera es la estructura base de cualquier programa interactivo.
- Git guarda versiones de tu código; GitHub las comparte.
- El ciclo de entrega (`init` → `add` → `commit` → `push`) se repite en cada TP del curso.

Lo que viene

En el **Encuentro 9 — Evaluación de la Unidad 1** vamos a hacer una evaluación individual de la Unidad 1. Vas a tener que demostrar todo lo que aprendiste hasta acá: escribir un programa que use entrada, condicionales y bucles sin ayuda del grupo.

8. Errores comunes y trampas

Error	Causa	Fix
<code>git push</code> pide usuario y contraseña todo el tiempo	No se configuró la autenticación	Usar token de acceso personal (PAT) de GitHub en vez de contraseña
<code>.gitignore</code> mal escrito: <code>__pycache__</code> sin barra	Sin la barra, Git ignora el archivo <code>__pycache__</code> pero no la carpeta	Escribir <code>__pycache__</code> (con barra) para carpetas
<code>git add .</code> antes de crear <code>.gitignore</code>	El <code>__pycache__</code> ya quedó rastreado	Agregar <code>.gitignore</code> primero, después hacer <code>git rm -r --cached __pycache__</code>
El menú no vuelve a aparecer después de una opción	Faltó el bucle <code>while</code> principal	Envolver el menú en <code>while running:</code> y solo salir con <code>running = False</code>
Descuento mal aplicado: <code>precio * 0.9</code> pero el usuario ve decimales infinitos	No se formateó la salida	Mostrar con <code>:.2f</code> en la f-string
<code>git commit</code> sin <code>-m</code> abre un editor de texto	Git abre el editor por defecto si no se pasa mensaje	Siempre usar <code>git commit -m "mensaje"</code>

Evaluación de la Unidad 1 — Encuentro 9

Evaluación de la instancia «Evaluación de la Unidad 1 — Encuentro 9» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-u1-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-u1-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación de la Unidad 1 — Encuentro 9
Unidad evaluada	1 — Fundamentos de Python y control del flujo
Eje temático	1 — Fundamentos del lenguaje Python
Carácter/Objetivo	Evaluación práctica individual: verificar la capacidad de escribir un programa de consola que combine entrada, conversión de tipos, condicionales y bucles.
Destinatarios	Todo el curso
Duración teórica	120 minutos (2 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-u1.md</code> · <code>evaluacion-u1-version-a.md</code> · <code>evaluacion-u1-version-a-anexo-docente.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (120 min)

Momento	Tiempo	Qué ocurre
Apertura e instrucciones	10 min	El docente explica la dinámica del encuentro: defensa del TP por grupo + prueba individual. Asigna versiones (A o B) por fila.
Defensa individual del TP (grupos)	30 min	Mientras un grupo rinde la prueba, el docente toma la defensa del TP a otro grupo en rotación.
Prueba práctica individual	70 min	Cada estudiante resuelve su ejercicio de forma individual en su computadora.
Cierre y recogida	10 min	Los estudiantes entregan por Git (commit + push). El docente cierra el encuentro y anuncia la devolución para el encuentro siguiente.
Total	120 min	

3. Regla canónica de la instancia

La entrega del TP-U1 se verificó en el encuentro 8 (cierre de unidad). En este encuentro 9 se realiza la defensa individual del

TP y la prueba práctica individual escrita. La devolución de ambos se realiza al inicio del encuentro siguiente (encuentro 10). Si un estudiante no completa la prueba en el tiempo asignado, entrega lo que tenga y se evalúa sobre lo entregado; no hay recuperación intra-encuentro.

4. Defensa individual del TP (modalidad)

Mientras un grupo rinde la prueba práctica, el docente se reúne por separado con cada grupo y verifica:

1. Que el repositorio existe en GitHub con la carpeta `tp-u1/`.
2. Que el archivo `kiosco.py` está presente y ejecutable.
3. Demo rápida: el grupo ejecuta el programa y muestra las 4 opciones del menú.
4. Pregunta individual a cada integrante sobre una parte del código (p. ej., «¿qué hace este `while`?», «¿por qué usaron `try/except` acá?»).
5. Registro en planilla: cada objetivo del TP se marca como logrado / parcial / no logrado.

5. Alcance

Incluido: variables y tipos (`int`, `float`, `str`), `input()` y `print()`, conversión con `int()` / `float()`, f-strings, condicionales `if/elif/else`, operadores de comparación y lógicos, bucle `while` con bandera, contadores y acumuladores, validación con `try/except ValueError`.

Excluido: colecciones (`list`, `tuple`, `set`, `dict`) como tema principal; funciones `def`; archivos; POO.

Objetivo mínimo: escribir un programa de consola funcional que lea datos del usuario, tome decisiones con `if/elif/else` y repita con un bucle `while`, manejando entradas inválidas con `try/except`.

6. Prueba práctica individual (versiones equivalentes)

La prueba consiste en **un único ejercicio práctico** que se resuelve en un archivo `.py`. Puntaje total: 100 puntos. El enunciado se organiza en partes que reflejan secciones del mismo programa:

- **Parte 1 — Menú y visualización (30 pts):** diccionario de productos provisto, menú con `while`, opción de listar productos.
- **Parte 2 — Registro de venta (40 pts):** entrada con validación, condicionales para stock y descuento, actualización de acumuladores.
- **Parte 3 — Resumen de ventas (20 pts):** mostrar total de unidades vendidas y dinero acumulado.
- **Parte 4 — Ítems conceptuales (10 pts):** preguntas breves sobre conceptos de la unidad.

7. Criterios de calificación

Componente	Puntaje	Qué se observa
Estructura y convenciones	10 pts	Un solo archivo <code>.py</code> , funciones arriba (si usa) o código limpio, bloque <code>if __name__ == "__main__":</code> , f-strings, identificadores en inglés snake_case, mensajes en español.
Funcionamiento del menú	20 pts	El menú se repite hasta elegir Salir; las opciones válidas ejecutan su acción; las inválidas muestran mensaje claro.
Registro de venta	40 pts	Lectura de código y cantidad con validación <code>try/except</code> , verificación de existencia del producto y stock suficiente, descuento condicional, actualización de stock y acumuladores.
Resumen de ventas	10 pts	Muestra unidades totales y dinero acumulado correctamente.
Ítems conceptuales	10 pts	Respuestas correctas a preguntas sobre tipos, conversión y bucles.
Defensa del TP	10 pts	Repositorio en orden, demo funcional, respuesta individual del integrante.
Total	100 pts	

8. Condiciones de resolución de la prueba

- **Individual:** cada estudiante resuelve su propia prueba en su computadora.
- **Material consultable:** apuntes propios, clases del aula virtual, documentación de Python. No se permite comunicación entre estudiantes.
- **Convenciones obligatorias:** las de `input/materias/LAP/convenciones-tecnicas.md` (snake_case, f-strings, `if __name__`, sin persistencia, sin `except:` desnudo).
- **Entrega:** commit + push al repositorio grupal en la carpeta `evaluacion-u1/` antes de que termine el encuentro. Si no hay push, se evalúa sobre el archivo local que el estudiante muestre al docente.

9. Regla de equivalencia entre versiones

Las versiones A y B comparten la misma estructura, los mismos objetivos de aprendizaje y los mismos requisitos. Cambia exclusivamente el dominio de aplicación (kiosco escolar ↔ biblioteca de aula) y los tokens de identificadores y datos de ejemplo que lo reflejan. Ninguna versión tiene reglas adicionales que la otra no tenga. Ambas se corrigen con la misma tabla de puntaje.

10. Mecánica de asignación de versiones

El docente asigna las versiones por fila de la sala: fila impar → versión A (kiosco), fila par → versión B (biblioteca). Se registra la versión asignada en la planilla de curso junto con el puntaje obtenido.

11. Devolución

La devolución se realiza al inicio del encuentro siguiente (encuentro 10). El docente entrega la planilla con puntajes y comentarios breves por criterio. Los estudiantes que no alcancen el objetivo mínimo (menos de 60 puntos en la prueba o TP no defendido) reciben una pista de recuperación: deben rehacer el ejercicio en su versión corregida y presentarlo al inicio del encuentro 11. El docente lleva registro en la planilla de seguimiento.

Evaluación de la Unidad 1 — Encuentro 9 — Versión A

Dominio de esta versión: **kiosco escolar** (venta de artículos, stock y descuentos). Duración: 70 minutos (bloque de resolución). Puntaje total: 100 puntos. Resolución individual, con computadora, sin celular. Las condiciones completas están en `evaluacion-u1.md`.

Antes de empezar

- Creá una carpeta `evaluacion-u1/` dentro de tu repositorio grupal.
- Dentro de esa carpeta, creá un único archivo `kiosco.py`.
- El programa debe seguir las **convenciones del curso**: identificadores en inglés y `snake_case`; comentarios y mensajes al usuario en español; f-strings para la salida; sin `except:` desnudo; sin archivos ni persistencia.
- Toda entrada numérica debe validarse con `while True` + `try/except ValueError`, mostrando un mensaje accionable en español.
- Al terminar, hacé commit con `git commit -m "evaluacion-u1: entrega"` y push al repositorio remoto.

Objetivos de la prueba

1. Escribir un programa de consola que use variables, condicionales y bucles.
2. Validar entrada del usuario con `try/except ValueError`.
3. Aplicar descuentos condicionales y acumular totales.
4. Organizar el código con funciones arriba y bloque `if __name__ == "__main__":` al final.

Material provisto

No hay esqueleto inicial. El estudiante escribe el programa completo desde cero. Se espera que use un diccionario para los artículos del kiosco.

Parte 1 — Menú principal y listado de artículos (30 puntos)

Agregá un diccionario `products` con al menos 4 artículos del kiosco. Cada clave es un código numérico (`int`) y cada valor es un diccionario con `name`, `price` y `stock`.

Mostrá un menú con las opciones:

Opción	Acción	Pts
1	Ver artículos: mostrará código, nombre, precio y stock de cada producto	10
2	Registrar una venta	— (se evalúa en Parte 2)
3	Mostrar resumen de ventas	— (se evalúa en Parte 3)
4	Salir	—
<i>inválida</i>	Mostrar “Opción inválida. Intentá de nuevo.”	10
<i>bucle</i>	El menú se repite hasta que el usuario elige Salir	10

El programa debe repetir el menú hasta que el usuario elige “4 — Salir”. Usá un bucle `while` con una bandera (`running = True` / `running = False`).

Parte 2 — Registrar una venta (40 puntos)

Cuando el usuario elige la opción 2, el programa debe:

Ítem	Consigna	Pts
2a	Pedir un código de producto con <code>input()</code> . Validar que el código sea un número entero.	10
2b	Verificar que el código exista en <code>products</code> . Si no, mostrar “Código inválido. No existe ese producto.” y volver al menú.	10
2c	Pedir la cantidad a comprar. Validar que sea entero positivo.	5
2d	Verificar que haya stock suficiente. Si no, mostrar “Stock insuficiente. Stock actual: {itemStock}.” y volver al menú.	5
2e	Si la cantidad supera las 5 unidades, aplicar un <code>discountRate</code> del 10 % al <code>itemPrice</code> . Mostrar el <code>saleTotal</code> con descuento detallado.	5
2f	Descontar del stock y acumular en variables globales <code>total_units</code> y <code>total_money</code> .	5
Total Parte 2		40

Parte 3 — Resumen de ventas (20 puntos)

Cuando el usuario elige la opción 3, el programa debe mostrar:

Ítem	Consigna	Pts
3a	Total de unidades vendidas en la sesión	10
3b	Dinero acumulado en la sesión, formateado con 2 decimales	10
Total Parte 3		20

Parte 4 — Ítems conceptuales (10 puntos)

Respondé brevemente en comentarios al final del archivo (después del bloque `if __name__ == "__main__":`):

Ítem	Pregunta	Pts
4a	¿Qué tipo de dato devuelve <code>input()</code> siempre? ¿Por qué hay que convertirlo antes de usarlo como número?	5
4b	¿Qué pasa si un <code>cliente</code> elige una opción de menú que no existe? ¿Dónde se maneja eso en tu programa?	5
Total Parte 4		10

Batería de verificación: salida esperada

Prueba	Pedido	Salida esperada (aproximada)
Menú	El programa arranca	Menú con 4 opciones, espera input
Opción 1	Ver lista	Muestra código, nombre, precio y stock de cada producto
Opción 2	Código 101, cantidad 3	Descuenta stock, no aplica descuento, muestra "Total: \$..."
Opción 2 (cantidad > 5)	Código 101, cantidad 7	Aplica 10 % descuento, muestra "Subtotal: \$..., Descuento: \$..., Total: \$..."
Opción 2 (stock insuf.)	Código 101, cantidad 999	"Stock insuficiente. Stock actual: ..."
Opción 3	Mostrar resumen	"Unidades vendidas: ... - Dinero acumulado: \$..."
Opción 4	Salir	"Saludos!" y termina
Opción inválida	"9"	"Opción inválida. Intentá de nuevo."
Entrada no numérica	"abc" en código	"Eso no es un número entero; intentá de nuevo."

Al terminar

- Verificá que el archivo se ejecuta sin errores: `python kiosco.py`.
- Hacé `git add .`, `git commit -m "evaluacion-u1: entrega"` y `git push`.
- Si no podés hacer push, avisá al docente antes de que termine el encuentro.

Evaluación de la Unidad 1 — Encuentro 9 — Versión A

Dominio de esta versión: **biblioteca escolar** (préstamo de libros, ejemplares y descuentos). Duración: 70 minutos (bloque de resolución). Puntaje total: 100 puntos. Resolución individual, con computadora, sin celular. Las condiciones completas están en `evaluacion-u1.md`.

Antes de empezar

- Creá una carpeta `evaluacion-u1/` dentro de tu repositorio grupal.
- Dentro de esa carpeta, creá un único archivo `biblioteca.py`.
- El programa debe seguir las **convenciones del curso**: identificadores en inglés y `snake_case`; comentarios y mensajes al usuario en español; f-strings para la salida; sin `except:` desnudo; sin archivos ni persistencia.
- Toda entrada numérica debe validarse con `while True` + `try/except ValueError`, mostrando un mensaje accionable en español.
- Al terminar, hacé commit con `git commit -m "evaluacion-u1: entrega"` y push al repositorio remoto.

Objetivos de la prueba

1. Escribir un programa de consola que use variables, condicionales y bucles.
2. Validar entrada del usuario con `try/except ValueError`.
3. Aplicar descuentos condicionales y acumular totales.
4. Organizar el código con funciones arriba y bloque `if __name__ == "__main__":` al final.

Material provisto

No hay esqueleto inicial. El estudiante escribe el programa completo desde cero. Se espera que use un diccionario para los libros del biblioteca.

Parte 1 — Menú principal y listado de libros (30 puntos)

Agregá un diccionario `products` con al menos 4 libros del biblioteca. Cada clave es un código numérico (`int`) y cada valor es un diccionario con `name`, `price` y `ejemplares`.

Mostrá un menú con las opciones:

Opción	Acción	Pts
1	Ver libros: mostrá código, nombre, tarifa y ejemplares de cada película	10
2	Registrar una préstamo	— (se evalúa en Parte 2)
3	Mostrar resumen de ventas	— (se evalúa en Parte 3)
4	Salir	—
<i>inválida</i>	Mostrar “Opción inválida. Intentá de nuevo.”	10
<i>bucle</i>	El menú se repite hasta que el usuario elige Salir	10

El programa debe repetir el menú hasta que el usuario elige “4 — Salir”. Usá un bucle `while` con una bandera (`running = True` / `running = False`).

Parte 2 — Registrar una préstamo (40 puntos)

Cuando el usuario elige la opción 2, el programa debe:

Ítem	Consigna	Pts
2a	Pedir un código de película con <code>input()</code> . Validar que el código sea un número entero.	10
2b	Verificar que el código exista en <code>products</code> . Si no, mostrar “Código inválido. No existe ese película.” y volver al menú.	10
2c	Pedir la cantidad a comprar. Validar que sea entero positivo.	5
2d	Verificar que haya ejemplares suficiente. Si no, mostrar “Ejemplares insuficiente. Ejemplares actual: {bookCopies}.” y volver al menú.	5
2e	Si la cantidad supera las 5 unidades, aplicar un <code>lateFee</code> del 10 % al <code>bookPrice</code> . Mostrar el <code>loanTotal</code> con recargo detallado.	5
2f	Descontar del ejemplares y acumular en variables globales <code>total_units</code> y <code>total_money</code> .	5
Total Parte 2		40

Parte 3 — Resumen de ventas (20 puntos)

Cuando el usuario elige la opción 3, el programa debe mostrar:

Ítem	Consigna	Pts
3a	Total de unidades vendidas en la sesión	10
3b	Dinero acumulado en la sesión, formateado con 2 decimales	10
Total Parte 3		20

Parte 4 — Ítems conceptuales (10 puntos)

Respondé brevemente en comentarios al final del archivo (después del bloque `if __name__ == "__main__":`):

Ítem	Pregunta	Pts
4a	¿Qué tipo de dato devuelve <code>input()</code> siempre? ¿Por qué hay que convertirlo antes de usarlo como número?	5
4b	¿Qué pasa si un <code>lector</code> elige una opción de menú que no existe? ¿Dónde se maneja eso en tu programa?	5
Total Parte 4		10

Batería de verificación: salida esperada

Prueba	Turno	Salida esperada (aproximada)
Menú	El programa arranca	Menú con 4 opciones, espera input
Opción 1	Ver lista	Muestra código, nombre, tarifa y ejemplares de cada película
Opción 2	Código 101, cantidad 3	Descuenta ejemplares, no aplica recargo, muestra "Total: \$..."
Opción 2 (cantidad > 5)	Código 101, cantidad 7	Aplica 10 % recargo, muestra "Subtotal: \$..., Recargo: \$..., Total: \$..."
Opción 2 (ejemplares insuf.)	Código 101, cantidad 999	"Ejemplares insuficiente. Ejemplares actual: ..."
Opción 3	Mostrar resumen	"Unidades vendidas: ... - Dinero acumulado: \$..."
Opción 4	Salir	"Saludos!" y termina
Opción inválida	"g"	"Opción inválida. Intentá de nuevo."
Entrada no numérica	"abc" en código	"Eso no es un número entero; intentá de nuevo."

Al terminar

- Verificá que el archivo se ejecuta sin errores: `python biblioteca.py`.
- Hacé `git add .`, `git commit -m "evaluacion-ul: entrega"` y `git push`.
- Si no podés hacer push, avisá al docente antes de que termine el encuentro.